

GenRTC: Enabling Generative Visual Captions in Live Video Streaming

Anonymous Author(s)

Abstract

Visual captions enhance the understanding of complex or unfamiliar concepts in live video streaming, such as TV broadcasts or video conferences. The process typically involves two steps: proposing an appropriate image based on user speech and seamlessly integrating it into the video. For live streaming, visual captioning must operate in real-time, completing both steps for one concept before the next arises. However, these steps demand intensive computational resources. This paper proposes GenRTC, a real-time visual captioning system comprising three key components. First, we leverage LLMs to predict user-referenced concepts and pre-generate corresponding visual captions. Second, a codec-aware scheduler integrates captions into the video stream adhering to the WebRTC protocol. Third, to support multiple simultaneous video streams, GenRTC incorporates a dynamic computational resource management scheme, optimizing the execution of visual captioning tasks across all streams. Data-driven simulations and real-world experiments demonstrate GenRTC delivers high prediction accuracy and maintains excellent video quality, ensuring a seamless live-streaming experience.

1 Introduction

Live video streaming plays a vital role in sectors such as web conferencing [53, 76], live TV streaming [65], cloud gaming [59, 62], interactive virtual reality (VR) [30], and IoT applications [60]. Visual captioning dynamically generates images to illustrate some complex or unusual concepts in conversations, improving viewers’ understanding. Figure 1 illustrates a live streaming example where the speaker introduces a computing platform. Generated visual captions make it easier for viewers unfamiliar with these domains to comprehend the specific scenarios being discussed.

While promising, aligning visual captions with a speaker’s content remains challenging. Since it is for live video streaming, we cannot prepare the visual captions beforehand. VisualCaptions [23] retrieves images from the Internet when the

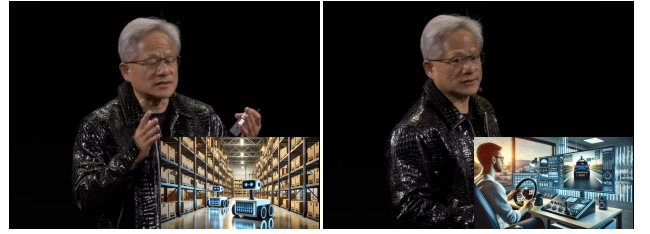


Figure 1: One example use cases of GenRTC for live streaming. Visual captions on smart robots and remote-controlled driving are generated when the speaker mentioned them.

speaker says a concept. However, it struggles with complex or uncommon scenes where it fails to deliver relevant images, potentially confusing the audience. This paper presents GenRTC, a visual captioning system that leverages advanced text-to-image generative models [1, 6, 40, 52], such as Stable Diffusion [44], to generate images directly from users’ words, enabling the visualization of unusual or complex scenes.

However, to integrate text-to-image generative models into live streaming systems, we need to tackle three key challenges. **First**, the generation process for visual captioning is time-consuming, making real-time visualization difficult; and the latency of rendering the images and the original video frames is long too, which makes the visual captions appear with a large delay after corresponding concepts mentioned, significantly disrupting the user experience. **Second**, integrating generated images into the default video streaming workflow introduces complications. The generation pipeline may interfere with video encoding, increasing frame delays and reducing video quality. In systems like WebRTC that use frame-dropping strategies to maintain synchronization, the computational load from visual generation can limit the framerate, affecting video fluidity and overall viewing experience. **Third**, the resource-intensive nature of real-time generative models often creates bottlenecks, particularly when server capacity is stretched by concurrent live streaming sessions. Managing computational resources (GPU Flops) on a server to optimize the global Quality of Experience (QoE) for all

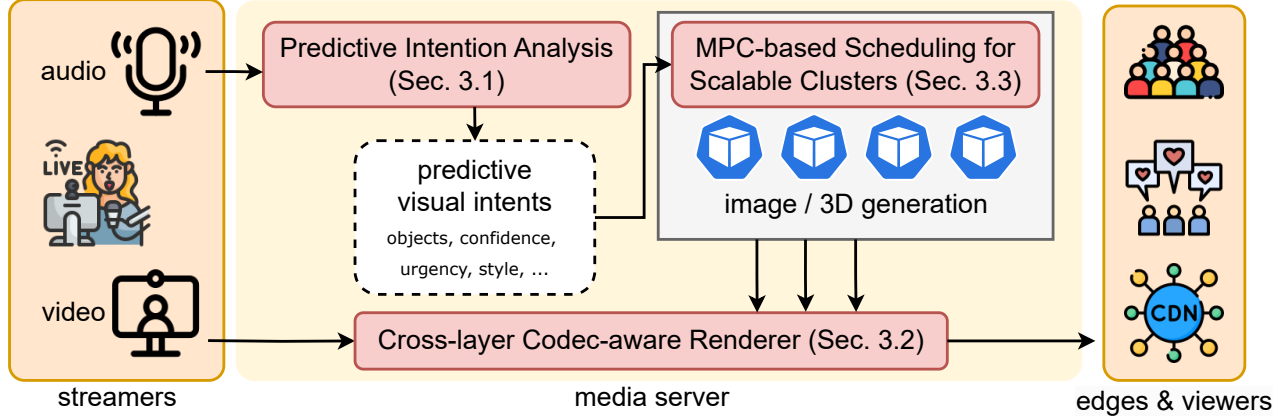


Figure 2: Workflow of GenRTC, including three components: intent analysis, intent rendering, and scheduling for scalable streamers. Intent refers to the visual content (e.g., images) that a speaker wishes to display.

users is critical. This requires advanced scheduling algorithms and resource allocation strategies to ensure QoE across all sessions without compromising visual captioning performance.

To overcome the above challenges, GenRTC is designed with three novel technical components, as shown in Figure 2. First, we introduce a **predictive intention analysis** module to perform audio-to-text conversion and fine-tune Large Language Models (LLMs) to discern current focal points or predict future intents in the dialogue. An intent is the visual content a speaker wishes to display on the streaming video, capturing her implicit desire to explain her spoken words. This proactive approach generates visual captions before a certain focal point is mentioned, mitigating the impact on user experience from the delays caused by time-consuming LLM inference. Next, we develop a **cross-layer codec-aware renderer** that integrates the generated images into the original video frames with minimum processing latency and video quality degradation. This module synchronizes the generative and rendering-encoding workflows, optimizing parallel processing to minimize the impact on user experience, effectively balancing the load between visual generation and video streaming tasks, and mitigating the interference of generation workload to the streaming workflow. Lastly, we design a resource allocation strategy for servers to ensure scalability across multiple live-streaming sessions. We formulate and solve an online bin-packing problem solved with Model Predictive Control (MPC) that enables efficient and responsive resource management by considering a set of factors, such as the urgency of visual intents, quality requirements, and network conditions. Additionally, video quality and visual captioning performance can be incrementally improved as more resources become available.

We implemented GenRTC with a widely used open-source real-time communication framework (WebRTC [59]). We tested different computation partitioning strategies and found the most efficient one, i.e., all video processing operations are handled on the media server. To validate GenRTC’s ef-

fectiveness, we conducted a comprehensive evaluation via trace-driven simulations and real-world experiments. We also assessed the impact of generative visual captions on system performance and user experience. The results show that GenRTC achieves an average perceived delay of 5.08 s, with a deduction of 2.13 s over the strawman implementation. 71.2% among all intents are rendered in time, which is $4.62\times$ over strawman. The overhead on frame latency and PSNR drop is 14.6ms and 0.19 dB, which are negligible. Case studies on live streaming and collaborative VR further validate the user experience provided by GenRTC.

To summarize, this paper has the following contributions:

- Proposing GenRTC, leveraging generative AI to perform visual captioning for live video streaming.
- Designing three system designs that reduce the perceived latency introduced by generation workloads, transforming GenRTC into a practical real-time visual captioning system.
- Evaluation indicates that the end-to-end latency, scalability, fairness, and user study are significantly improved and fulfill the user requirements.

2 Adopting Generative AI for Live Visual Captions: Benefits and Challenges

2.1 Benefits

Advanced text-to-image generative models [1, 6, 40, 52], such as Stable Diffusion [46], generate images from text descriptions, enabling visualization of complex or rare scenes that are often unavailable on the Web. To validate this proposal, we compared two approaches: retrieving images from the Internet and generating images using Stable Diffusion [46]. We use two categories of text prompts: text from TV clips where people are unaware of generative functionality and prompts collected from user studies where volunteers know

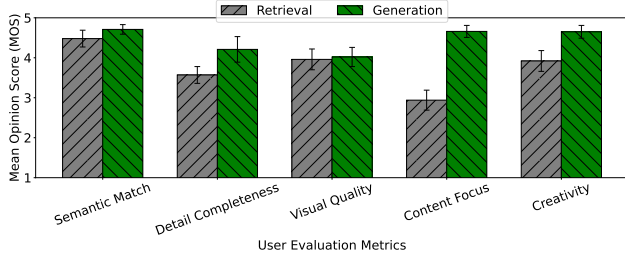


Figure 3: Generated visual captions improve user intent satisfaction significantly over retrieval.

about that their intents will be visualized. User scores are used to evaluate how well each image matches its corresponding text description, from 5 perspectives: semantic match, detail completeness, visual quality, content focus, and creativity, with each on a 5-point scale (1=very bad, 2=bad, 3=fair, 4=good, 5=very good).

As illustrated in Figure 3, A user study with 20 participants revealed significant improvements when using generative captions compared to retrieval-based results: Increasing semantic match improved by 5.1%, detail completeness by 17.9%, visual quality by 1.5%, content focus by 58.5%, and creativity by 18.6%. These results highlight the ability of generative models to provide clearer, more focused, and imaginative visual representations. By addressing the limitations of retrieval-based approaches, generative models enable transformative applications in education, creative design, and personalized content generation.

2.2 Challenges

While integrating generated visual captions into live video streaming offers significant benefits, it also introduces unique challenges. Existing works on live streaming has explored different aspects of system optimization, e.g., neural compression [5, 45, 52], adaptive frame rate [29] and redundancy reduction [28], which mainly focuses on reduce the overall latency. However, these approaches can not enable live visual captions due to the following challenges we recognized during the implementation:

2.2.1 Long Generation Time vs. Strict Requirements

When using generative models for visual captions, the process begins by extracting text descriptions as prompts from the speech. These prompts guide the models in generating corresponding images. However, the dynamic nature of topic migration presents alignment challenges.

Resource-Intensive Generative Models. Although text-to-image models such as DALL-E 2 [39], DALL-E 3 [40] and Stable Diffusion [44] produce high-quality images, inference is significantly time-consuming. Among these advanced models in our test on an A100 GPU, the fastest average inference

record is 3.81s by SD. This record does not include network delays or the time required for intent recognition.

Frequent Intent Migration in Verbal Communication.

Human speech typically occurs at a pace of 100-150 words per minute in English [10], which allows speakers to shift rapidly across topics [25]. In those cases, each visual intent is set a strict deadline for the generated visuals to be rendered, which is the end of this topic or the start of the new one. Otherwise, they will be outdated and confuse the audience with misaligned captions. In this case, they should only be discarded and computing resources are wasted. In our dataset, the average intent duration is 5.17 seconds, with 21.9% of intents lasting less than 3.81 seconds.

This misalignment makes it impractical to incorporate generative functionalities for commercial applications.

2.2.2 Video Quality Degradation

During implementation, we observed that integrating visual captions into live video streams could build up the overall latency and degrade the overall quality of the original video.

For local rendering, the delay does not impact the original video, as it can be deferred, similar to auto-captions on commercial platforms like YouTube [69]. However, server-side real-time rendering is different because the entire video must be synchronized before transmission. The reasons for latency buildup are two-fold. First, the video must be decoded for rendering and then re-encoded, introducing additional latency. Second, the generation and rendering processes increase CPU and memory usage, further delaying video processing [55]. In our measurement, the median RTT increased to 265 ms. However, for interactive video streaming systems, end-to-end latency is expected to remain below 130 ms [14, 27, 28].

Moreover, the additional jitter caused by these delays leads to uneven video delivery, which can cause the transport layer protocol to overestimate queueing delays and, as a result, reduce the bitrate selection. This, combined with fluctuating network conditions between participants, exacerbates performance issues and impacts the consistency of video quality. Quantitative measures in PSNR also show significant drops, as illustrated in Figure 9c, highlighting the need to mitigate the impact of additional computation workloads.

The generation pipeline introduces frame drop and quality degradation. Specifically, the rendering process causes frame drops in WebRTC’s frame logic. In VoD scenarios with a large buffer, rendering does not affect quality. However, in real-time streaming, without an explicit buffer, Google Congestion Control (GCC) [3] perceives any frame delay as network congestion, leading to a reduction in bitrate. As illustrated in Figure 4, the jitter caused by the rendering delay will cause the receiver side to send negative feedback through RTCP packets. The sender side mistakenly takes it as network fluctuations, resulting in conservative bitrate adjustments.

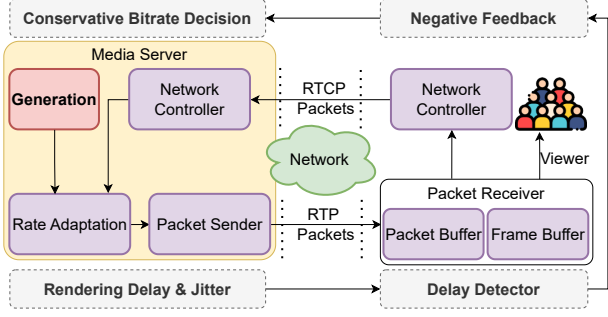


Figure 4: Illustration of how the generation process causes frame delay during rate adaptations.

2.2.3 Scaling up to Multi-session

Commercial streaming providers commonly use CDNs to manage downstream traffic [4, 66], while central media servers handle millions of simultaneous streams using advanced signaling techniques [61]. This infrastructure is designed to deliver high-quality streaming experiences to large user bases.

Scaling the generation of visual captions across multiple sessions, however, presents challenges. Each session varies in network conditions, object distributions, and quality or urgency requirements, complicating scheduling and resource allocation. To address these challenges, the system must dynamically adapt to real-time demands, ensuring efficient resource usage while maintaining overall system performance.

The strawman’s task scheduling is managed using a first-fit (FF) algorithm [18], where the available GPU pods are assigned tasks immediately. Additional tasks are placed in a queue until resources become available. Missed intents, whether from pods or the queue, are discarded to prevent outdated requests from consuming resources, ensuring efficient processing and timely visual captions.

Throughout our evaluation, the strawman solution achieves an intent rendering rate of only 15.1%, highlighting a significant under-utilization of computational resources. For the 84.9% of missed intents, many of them have halfway inference but have become outdated. This inefficiency is primarily due to rapid shifts in user intent and varying intent requirements across sessions. These factors contribute to delays in visual caption generation, often resulting in missed objects and suboptimal user experiences.

3 Design of GenRTC

In this section, we detail the design of GenRTC describing the three key modules shown in Figure 2. The intention analysis and object planning module continuously parses speech and outputs the objects the speaker intends to visualize. The cross-layer codec-aware renderer ensures optimal video quality and latency during rendering. Finally, the scalable clusters and scheduler maximize GPU resource utilization when multiple sessions are hosted on a single media server.

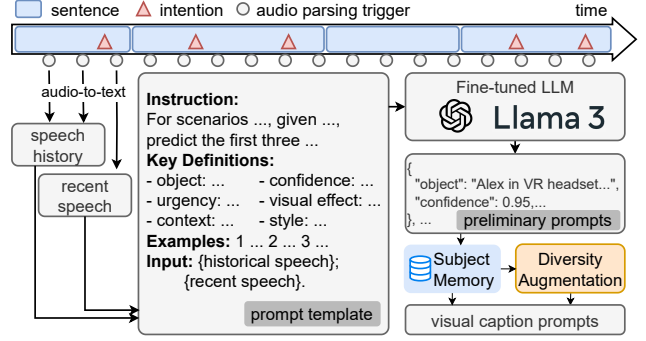


Figure 5: Framework for analyzing user intents.

3.1 Predictive Intention Analysis

GenRTC employs audio-to-text models and fine-tuned LLMs to perform real-time speech analysis during video streams.

Figure 5 illustrates this process, where each gray circle represents audio parsing triggered at regular intervals. The speech history and recent speech are organized into a few-shot prompt, allowing fine-tuned LLMs to identify and describe the three intents most likely needed by the speaker. Specifically, the prompt template guides the LLM to output in a JSON format, including the description and confidence level. In some instances, if no new intent is detected in the recent speech, the output visual caption prompts remain unchanged.

Additionally, to ensure visual consistency, we maintain a subject memory that stores visual feature descriptions of intents. After generating the initial prompts, the subject memory complements them and updates itself based on new information. This approach guarantees visual consistency [34, 43]; for example, the visual representation of ‘Alex’ remains consistent across multiple samples. For human subjects without prior memory, we apply diversity augmentation [20] to ensure diversity in gender, age, and other traits.

LLM Fine-Tuning. We fine-tune LLMs with labeled datasets to improve accuracy, using frameworks from Together.ai for seamless integration of custom data into pre-trained models.

The dataset consists of 263 manually labeled ($\langle \text{speech text} \rangle$, $\langle \text{intent} \rangle$) pairs, mapping speech segments to corresponding intents. LLaMA-3 70B [31] serves as base models, fine-tuned through multiple iterations to better align with the dataset. Fine-tuned LLaMA-3 improves intent analysis accuracy by 21.4% and Top-3 accuracy by 17.1%. These results highlight an enhanced understanding of speech-intent relationships, leading to more accurate, contextually relevant predictions in live video streaming.

3.2 Cross-Layer Codec-Aware Renderer

Traditional media servers simply forward media content without modification. However, to enhance user experience, we enrich media streams by rendering visual captions into video

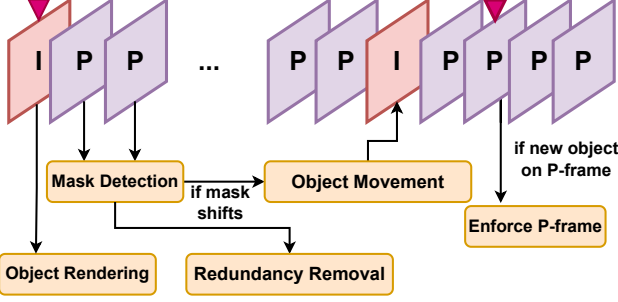


Figure 6: Codec-aware rendering, with the red triangle representing the rendering operation.

frames. To avoid heavy computational load of decoding and encoding in the strawman solution described in Section 2.2.2, we design a codec-aware, cross-layer renderer that efficiently integrates visual captions into WebRTC streams. Additionally, integrating visual captions into video frames requires modifying the GCC transport layer algorithm to ensure rate adaptation accounts for changes in frame size.

3.2.1 Codec-Aware Rendering Algorithm

Efficient live video streaming relies on video compression techniques, where the video is encoded on the streamer side and decoded on the viewer side. On the media server, while it is possible to decode, modify each frame, and re-encode it, this process introduces significant additional time consumption due to the overhead of these steps. Therefore, we introduce a codec-aware rendering algorithm to bypass this overhead by directly processing encoded frames.

Video codecs leverage inter-frame relationships, such as those between I-frames and predicted frames (P-frames). I-frames are full-size, self-contained frames that can be decoded independently and serve as key reference points. P-frames, in contrast, store only the differences from preceding frames, significantly reducing file sizes while maintaining continuity. P-frames are dynamically selected by the encoder based on video dynamics and application latency requirements. In live streaming encoding, the percentage of P-frames is high to minimize overall video file size [5], making P-frames optimization essential.

Figure 6 illustrates our algorithm for modifying encoded I-frames and P-frames to maintain high-quality visual communication under varying network conditions.

For I-frames, they directly render the visual captions on the default mask area. P-frames, on the other hand, are carefully processed, which first pass the mask detection, adaptive rendering and redundancy removal to optimize the efficiency.

Mask Area Detection. Mask detection identifies and updates the relatively static regions of the video frame, referred to as the mask area. This ensures that captions do not obstruct dynamic visual content, which is highly possible to be the viewer’s interests [71]. While computer vision algorithms could analyze each frame for static regions, they require sig-

nificant computational resources [64]. Instead, we adopt a more efficient method by leveraging the intrinsic information in P-frames. Since P-frames encode only the differences from preceding I-frames or P-frames, they naturally highlight areas of motion or change, eliminating the need for full-frame analysis. By analyzing the motion vectors encoded in P-frames, which represent the degree of change in different regions, static areas with low or zero motion can be identified. Every timestamp, we sample the next available P-frame and heuristically search for low-dynamic regions, designating them as the mask area for rendering visual captions without disrupting the scene’s flow. This approach reduces computational overhead and ensures captions are placed in unobtrusive areas, preserving both video quality and user experience.

If the mask area maintains, the visual caption should remain static. But if the mask area shifts due to dynamic content in subsequent frames, it will be calculated using the pixel density statistics of all P-frames between two I-frames and enforce the area change at the next I-frame.

Adaptive rendering. When a new visual caption comes on the timestamp of a P-frame, the module modifies this P-frame directly, ensuring visual captions remain aligned and relevant. The other P-frames will remain unchanged.

Redundancy Removal. For all P-frames, redundancy removal is performed at the end to avoid duplicating visual information. The intuition is that the original pixels covered by the visual captions no longer need to be transmitted. In the workflow, we directly filter out the redundant pixels without transmitting them.

This combination of mask detection, object rendering, and redundancy removal allows for smooth visual integration across both I-frames and P-frames, adapting dynamically to changes in the video stream.

3.2.2 Incorporating with Rate Adaptation

Rate adaptation adjusts the video bitrate based on available bandwidth, regulating I-frames and P-frames to maintain smooth streaming, even with limited bandwidth, by compressing frames and minimizing interruptions. This design is compatible with both H.264 and VP8 codecs.

Bitrate Adaptation. Algorithm 1 outlines the bitrate adaptation process. When modifying encoded frames, GenRTC employs a model predictive control (MPC) approach to manage frame rate preferences and P-frame queue sizes. This predictive model tests and adjusts frame-dropping ratios for the upcoming seconds of the stream, optimizing for: 1) *Ensuring the tunnel is not idle*. By dynamically adjusting the frame rate and size, the system ensures efficient use of the network tunnel, avoiding under-utilization that could cause perceived lag. 2) *Maximizing expected QoE*. The goal is to maximize user experience by balancing frame rate, resolution, and quality with available network bandwidth and the client device’s computational power. Resolution and frame rate are

Algorithm 1: Rate Adaptation MPC

Input: Past Latency, Network conditions, QoE metrics
Output: Bitrate calibration for the current state

```
1 Function AdaptationMPC (Latency, Network, QoE) :  
2   FrameRatePrefs  $\leftarrow$  initializePreferences()  
3   PFrameQueue  $\leftarrow$  initializePFrameQueue()  
4   while streaming do  
5     DropRatio  $\leftarrow$  predictRatio(Latency, Network)  
6     adjustFrameDrop(DropRatio)  
7     if tunnel is idle then  
8       | ensureNonIdleTunnel()  
9     end  
10    MaxQoE  $\leftarrow$  maximizeQoE(QoE, Network)  
11  end  
12  return calibrated bitrate
```

increased as much as possible, while reserving bandwidth to handle network fluctuations and variations in video frame size, ensuring a smoother experience for users.

Frame Rate Adaptation. To handle outdated frames, WebRTC provides the `DropTail` [37] functionality, which drops frames in the receiver queue and requests a new keyframe if overflow occurs. We use the same `DropTail` setting as a backup when jitter causes overflow.

3.3 MPC-Based Scalable Scheduling

For large-scale streaming services, ensuring scalability is crucial, as the media server must support multiple streaming sessions. In our system, GPU resources are utilized for generative model inference to create visuals, such as images. The GPU scheduling problem is formulated as an online knapsack problem [16, 61]. Specifically, intent-rendering tasks are queued and face two options: execute immediately or wait. The optimization objective is to maximize task completion (task satisfaction) within the available GPU capacity, while considering the urgency and confidence of each task.

1) Scheduling Input - Sequential Scheduling Task. We denote $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ as the set of intent-rendering tasks that arrive over time. Each task T_i has the following attributes:

- Priority $v_i \in \mathbb{R}^+$: representing the priority of the task.
- Confidence $c_i \in (0, 1]$: representing the resources required (e.g., GPU time) to complete the task.
- Intent Duration $d_i \in [0, 30000]$ (ms): representing the time period during which rendering is considered effective.
- Urgency $u_i \in [0, 30000]$ (ms): the estimated time until the task is needed, with 0 indicating immediate need.
- Roundtrip Latency $rtt_i \in \mathbb{R}^+$ (ms): measured by the WebRTC API and stored on the media server.

2) Scheduling Output - Decision Variable. A binary variable where $x_i = 1$ indicates that task T_i is selected for execution. Conversely, $x_i = 0$ indicates that task T_i is deferred, meaning it will not be executed at this time and may be revisited later depending on resources and scheduling priorities.

3) Optimization Objective. Maximize the total value of the executed tasks, subject to the C1 and C2 constraints:

$$\max \sum_{i=1}^n x_i \cdot v_i, \quad (1)$$

- C1: Higher-priority and higher-confidence tasks are favored, so the selection should consider urgency scores u_i , potentially integrated into the value term:

$$v_i = c_i \times \max \left(0, 1 - \frac{rtt_i + T_G}{d_i + u_i} \right) \quad (2)$$

where T_G is the mean generation time, and $1 - \frac{rtt_i + T_G}{d_i + u_i}$ represents the likelihood of meeting the deadline if the task is executed immediately.

- C2: Capacity Constraint: The total resources used by the selected tasks should not exceed the GPU's capacity C_{gpu} :

$$\sum_{i=1}^n x_i \cdot w_i \leq C_{gpu} \quad (3)$$

4) MPC-Based Solution. We propose an MPC-based scheduling algorithm to dynamically manage GPU resources. By predicting future task arrivals and estimating intent duration using the harmonic mean of the last 10 intents from the same streamer, the model adapts to the streamer's habits. The MPC algorithm evaluates tasks in real-time, considering urgency, priority, confidence, and GPU capacity to determine whether tasks should be executed immediately or deferred. It continuously re-evaluates decisions as new tasks enter the queue, ensuring efficient resource allocation and prioritization of high-priority tasks, while minimizing latency and adhering to GPU capacity constraints.

3.4 GenRTC: Put It Together

Figure 7 illustrates the GenRTC workflow for the generation and rendering during a video segment. The process begins with periodic intention analysis (IA), where the system listens to the speaker and predicts multiple possible intents based on the speech content. In this example, the speaker first mentions "Miami," triggering three intent predictions with varying confidence levels. GenRTC generates the first intent, which is rendered as an image and added to the video stream, indicated by the green triangle. The process continues with the second intent inferred and scheduled until the next IA event when predictions are updated. This design enables GenRTC to achieve fast and efficient rendering by processing multiple intents in parallel, minimizing latency, and ensuring that visual content remains synchronized with the speaker's dialogue.



Figure 7: An example of control workflow of GenRTC. Each line has different representations. "IA" stands for intention analysis. The blue circles indicate IA events that obtain updated results, while the white circles remain the same output as the previous one. The green triangle means successful rendering.

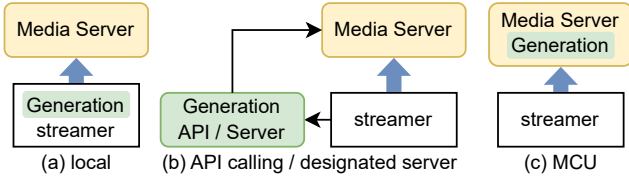


Figure 8: The major architectures of generative visual captioning in live video streaming. Blue arrows indicate heavy video streams while black arrows mean intermittent requests.

4 System Implementation

GenRTC is built on WebRTC, enabling real-time communication capabilities essential for live video streaming. We choose aiortc [2] for easier interfaces with Pytorch.

4.1 Client-Server Architecture

Figure 8 displays three major architectures, each differing in where the generative model is executed.

- (a) **Local generation**: Content generation is co-located with the streamer, enabling immediate processing and integration with streamed media. While this aligns well with the widely adopted SFU architecture, limited GPU capacity on streamers leads to high latency and overwhelmed CPUs [8], affecting video encoding and transmission. Experiments on an RTX 4090-equipped laptop show significant latency buildup and frame rate degradation, confirming its unsuitability for edge deployment.
- (b) **API calling / Designated server**: This scheme utilizes a separate server for generation tasks, interacting with the media server via an API, which helps distribute processing loads. However, the reliance on network connectivity between generation server and media server introduces latency issues, complicating real-time streaming performance.
- (c) **Multipoint Control Unit (MCU)**: The MCU architecture

involves putting the generation workload with the media server, similar to the traditional transcoding tasks. It is often selected for its capability to handle complex, stream processing tasks without the drawbacks mentioned above.

We measure the end-to-end latency of each scheme with the same network traces (Round-Trip Time (RTT) = 102 ms). The local machine is a laptop with an RTX 4090 GPU. For scheme (b), we use the API of calling stable diffusion instead of using closed-source models. The settings remain the same for the generation server and media server, but it also serves another streaming to maintain fairness. The average time gaps are 19,207 ms, 11,383 ms, and 9,604 ms, respectively. Therefore, the MCU architecture is the optimal solution for the best latency performance.

Note that we excluded placing generation workloads on the edges or viewers due to redundancy, that is, each device would need to generate its own visual captions, multiplying inference by the number of edges / clients.

Media Server. The server infrastructure of GenRTC is powered by high performance hardware, with NVIDIA A100 GPUs, designed to handle the intensive computational demands of real-time video processing and the generation of complex visual representations. The network interface card (NIC) specifications are tailored to support high-speed data transfer rates, essential for maintaining the quality and responsiveness of the video conferencing experience.

Streaming Client. Clients interact with the GenRTC system comprising a diverse range of devices, including laptops and Android phones, each accessing the service through web pages. This cross-platform compatibility ensures wide accessibility, catering to users in various scenarios. According to our measurements, the choice of end devices does not impact the QoE measurements. However, we include different types of devices in the case study.

GPU Scheduling over Container. The rendering process is encapsulated within Kubernetes containers, offering a mod-

ular organization that facilitates flexible GPU resource allocations and updates, which closely aligns with industrial practices. Within each worker node, Redis is used to manage the information of the media file, speech history and subject memory, and system status, ensuring efficient data retrieval and real-time updates.

Network Condition. To realistically simulate and optimize performance under typical usage scenarios, we evaluate both real network environments and trace-driven simulations. The real-world tests include both cellular and Wi-Fi connections at the edge. This approach addresses challenges posed by varying network speeds and stability, ensuring that GenRTC consistently delivers high-quality visualizations across different environments and network conditions.

5 Evaluation

In this section, we address the following questions through two open video datasets and two case studies:

- RQ1 How effective is GenRTC in analysing and predicting intents from the speech?
- RQ2 What’s the QoE compared with vanilla WebRTC and the strawman implementation?
- RQ3 Can GenRTC guarantee the performance across various video sources, network conditions, and devices?
- RQ4 Would the streamers and audiences prefer vanilla WebRTC or GenRTC?

5.1 Experimental Setting

We deploy a server with high-performance network interfaces, achieving an average upload speed of 920.79 Mbps. In real-world tests, clients use cellular and Wi-Fi, while for simulation purposes, we generate synthetic network traces to emulate the clients’ downlink connections. For generative models, we utilize Stable Diffusion v1.5 without further modifications.

Video Datasets. We use two video datasets featuring clear and continuous human speech in the audio. The first is Vox-Celeb [33], a large-scale audio-video dataset of human speech. The second consists of live TV clips from NBC, CBS, ABC *etc.*, collected from the Puffer [65]. These datasets provide a variety of real-world communication scenarios. All datasets were labeled with intents and timestamps by human-computer interaction experts. In total, 127 videos were labeled, with an average duration of 25 minutes and 11 seconds¹.

Baselines. To the best of our knowledge, GenRTC is the first work that incorporates generative visual captions in live

streaming.² To demonstrate the superior performance, we use the following three baselines:

- *Vanilla WebRTC without Visual Captions.* Vanilla WebRTC provides ideally uninterrupted streaming experience since no generation functionalities are applied. This serves as a skyline to demonstrate the sacrifice involved in incorporating visual caption generation. However, without visual captions, the overall user experience may lack the enhanced clarity and engagement that visual elements bring.
- *Strawman: An Intergrated SOTA.* To validate the performance improvement over the latest industry practices, we integrate VisualCaption [23], Kubernetes containers [16], and PIFO [51], contributing optimizations in pipeline efficiency, multi-session handling, and scheduling respectively.

Metrics. The following metrics assess system efficiency:

- *Intent accuracy.* Predictions from the fine-tuned LLM are compared with expert-labeled intents as the ground truth to evaluate intent prediction accuracy.
- *Perceived delay.* The delay is the combination of network latency and generation runtime. Therefore, the measured delay is $T_{up} + T_{gen} + T_{down}$, where T_{gen} represents delay that occurred on the media server.
- *Frame latency.* We measure the end-to-end delay of successfully received frames. As different applications pose varied deadline requirements, from 50 ms to 150ms. we choose 130 ms as a reference [27].
- *Video quality.* The peak signal-to-noise ratio (PSNR) is used to quantify the received video quality.
- *Intent Rendering Rate.* It measures the ratio of rendered captions to recognized intents. The next intent serves as the deadline for the current one. Failures include unprocessed intents or those that become outdated during generation, while successes are counted only when the intended prompts are successfully rendered. A higher ratio indicates that the system effectively generates and displays the correct visual content in real time across multiple streams.

These metrics exclude cases where GenRTC accurately predicts intents, but the rendered image does not align with the intent. This exclusion is justified for two reasons: First, modern text-to-image models generate high-quality images due to training on large-scale datasets [73]. These models consistently produce relevant, high-fidelity images, minimizing dissatisfaction related to intent alignment. Second, in our two case studies, participants were instructed to evaluate GenRTC, resulting in high satisfaction with the system.

²New fast models (e.g., ADD [48]) are not included as baselines, as they do not incorporate any system-wide designs. Instead, any improvements in model speed or quality can be seamlessly integrated into GenRTC.

¹We prefer long videos as they fully test the intent prediction functionality.

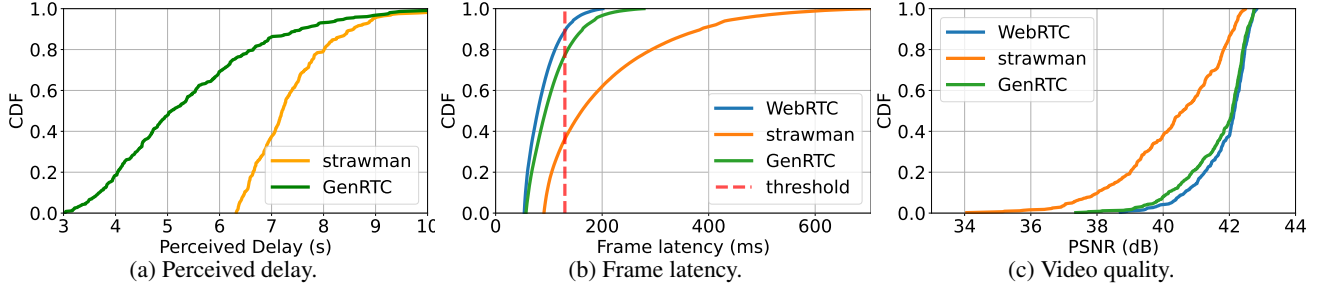


Figure 9: Overall performance on two combined video datasets for different methods with ten streamers.

	Resolution	GPUs	Network
#1	1280x720	A100	Ethernet
#2	854x480	2xA6000	Wi-Fi
#3	640x360	A6000	Cellular [7]
#4	426x240	3080 Ti	

Table 1: Settings in the generalizability study.

5.2 Overall Performance

All videos in the dataset are randomly sampled for the experiments in this section. Ten virtual streamers upload live video at the same time. An A100 GPU serves generation.

Perceived Delay. Figure 9a illustrates the perceived delay for both the *strawman* and GenRTC. Since the visual caption generation process takes about 4 seconds, the *strawman* implementation shows a median perceived delay of 7.21 seconds, which is due to its lack of optimization in predictive intent analysis and GPU resource scheduling. In contrast, GenRTC achieves a much lower median perceived delay of 5.08 seconds, demonstrating its more efficient handling of real-time caption generation and transmission. Moreover, GenRTC occasionally obtains perceived delay shorter than the inference time, thanks to the predictive intent analysis module.

Frame Latency. We compare the frame latency of GenRTC with two baselines. In Figure 9b, the median frame latency for *strawman* GenRTC is 257.91 ms, which is 2.67 times higher than the 79.22 ms of *Vanilla WebRTC*. Additionally, during continuous GPU full-duty periods, a noticeable long-tail distribution occurs, with 95-th and 99-th percentile latencies reaching 551.01 ms and 815.89 ms, respectively. However, the optimizations in GenRTC reduce the median latency to 93.78 ms, with the 95-th and 99-th percentile latencies dropping to 187.72 ms and 238.44 ms, respectively. This results in only an 18% increase compared to *Vanilla WebRTC*.

Video Quality. In Figure 9c, the video quality of three different methods is compared. *Vanilla WebRTC* provides the best video quality, serving as the benchmark with a mean PSNR of 41.87 dB. In contrast, the *strawman* implementation performs significantly worse, with a mean of 40.31 dB, primarily due to the lack of optimization in handling visual tasks. However, *GenRTC* demonstrates much better perfor-

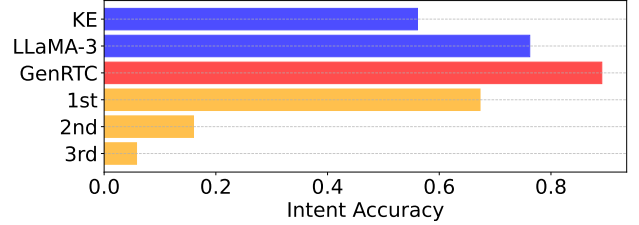


Figure 10: Overall performance of intention analysis.

mance, closely approaching the baseline with a mean of 41.68 dB. This improvement in GenRTC highlights the benefits of optimizations that mitigate the impact on video quality.

Intent Accuracy. Apart from LLMs, intent analysis can be performed using traditional NLP algorithms. For example, the keyword extraction algorithm [21] generates similar text that can be used as prompts. We use this as a baseline, referred to as *KE*. Figure 10 compares the Top-3 intent accuracy of *KE* (56.1%), vanilla LLaMA-3 (76.2%), and GenRTC (89.1%). For GenRTC, the accuracy breakdown across each output rank is 67.3%, 16.0%, and 5.8%, respectively. This result highlights the effectiveness of our fine-tuned LLMs in better predicting the speaker’s intent.

Intent Rendering Rate. We focus solely on whether the intent is generated and displayed on time, without considering correctness, as these depend on the performance of generative models. *GenRTC* achieves an intent rendering rate of 71.2%, while *vanilla WebRTC* reaches only 15.4%. This demonstrates that *GenRTC* significantly improves real-time intent rendering, due to its optimizations for handling visual captioning tasks. The detailed contribution of various factors to this difference is discussed in the ablation study in Sec 5.4.

Although the current intent rendering rate is optimized, a gap remains toward achieving full satisfaction. This is primarily due to sudden, short-lasting intents, such as a quick, novel topic that lasts for less than 3.81 seconds, which cannot be rendered in time. We attribute this limitation to current commodity hardware and believe that advances in MLPerf [38]³ will further enhance the integration of GenAI into live video streaming. For the 21.9% of intents that last less than 3.81 seconds, GenRTC still managed to render 14.0%, demonstrating the benefit of predictive intention analytics.

³NVIDIA reports 235.36 ms for SD image generation on H200 datacenters.

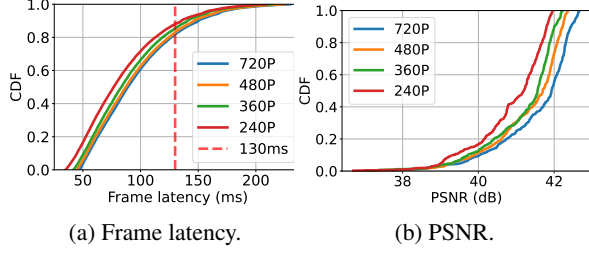


Figure 11: Impact of video resolutions.

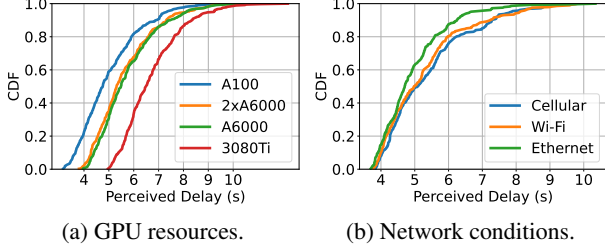


Figure 12: Impact of GPU resources and network conditions.

5.3 Generalizability Study

We evaluate the robustness and adaptability of the system under varying audio and video quality conditions. Table 1 details the different settings. This evaluation is crucial to ensure that GenRTC performs effectively in real-world scenarios with diverse streaming quality requirements.

Video Resolution. Most videos in Voxceleb dataset have a limited resolution of 256×256 . To rigorously test GenRTC’s capability to handle various video qualities, we utilize pre-recorded TV clips from Puffer [65]. The audio specifications do not affect QoE due to the minimal bitrate. To evaluate the generalizability of GenRTC with different video resolutions, we tested four levels: 1280×720 , 854×480 , 640×360 , and 426×240 . The results indicate that the impact on perceived delay is within 1%, which is negligible. As shown in Figures 11a and 11b, the frame latency and PSNR remain consistently high at all resolution levels.

GPU Resources. To assess the impact of GPU resources on QoE, we conduct comparison experiments using 1xA100, 2xA6000, 1xA6000, and 1xRTX 3080 Ti. The number of virtual streamers is still 10. The number of virtual streamers is set to 5 for this study. As shown in Figure 12a, the perceived delay generally follows the generation time. For the 2xA6000 setup, 14.5% of GPU time remains unused since we did not double the number of streamers. Additionally, we record the intent rendering rate for each setting, with the A100 being the highest because the minimum inference time is the lowest. Given that the rental cost of 1xA100 is similar to that of 2xA6000 [11, 17], we recommend prioritizing performance by using 1xA100 to enhance user experience and maximize GPU utilization. We observe that lower generation times lead to higher GPU utilization.

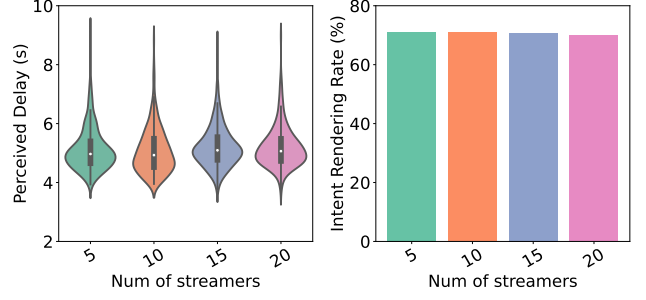


Figure 13: Perceived delay and intent rendering rate of GenRTC across varying numbers of streamers.

Network Condition. To evaluate whether GenRTC adapts to different network environments, we gathered two network traces using Mahimahi [36] for Ethernet and WiFi. FCC cellular traces [7] were also included. These traces capture variations in bandwidth, latency, and packet loss to simulate real-world scenarios. Performance is measured in terms of perceived delays. Figure 12b shows that network conditions have a negligible impact on perceived delay. GenRTC maintains high performance, with average perceived delays of 4948.6 ms on Ethernet, 5280.8 ms on WiFi, and 5385.0 ms on cellular, demonstrating robustness across various network conditions.

Number of Streamers. Figure 13 illustrates the scalability of GenRTC under varying loads. We set the number of streamers to 5, 10, 15, and 20. Each virtual streamer uploads a video file with a maximum resolution of 720p. The hardware setup serves 1, 2, 3, and 4 A100s GPUs for fairness. Note that with 5 users, the GPUs were not fully utilized, with 16.3% of duty time remaining idle.

5.4 Ablation Study

To validate the contribution of each component, we deactivate the following design components individually: continuous intention analysis and planning, codec-aware rendering scheduler, and dynamic resource allocation mechanism. This allows us to evaluate their respective contributions to overall system performance. We measure system latency from audio input to visual output, computational overhead through GPU and CPU usage, and visual fidelity based on the clarity and relevance of generated visuals in controlled user tests

Figure 14 presents the results of each ablated version. Replacing the continuous intention analysis module with a naive LLM increases the average perceived delay from 5.06s to 6.65s and drastically reduces intent rendering rate from 71.2% to 34.5%. Disabling the codec-aware rendering scheduler results in a slight increase in perceived delays and a 2.6% decrease in the intent rendering rate. Lastly, removing the dynamic resource allocation mechanism leads to suboptimal resource management during peak demands, causing perceived delay to rise by 46.2% and obtaining intent rendering delay as 55%. These findings affirm the critical role of each proposed

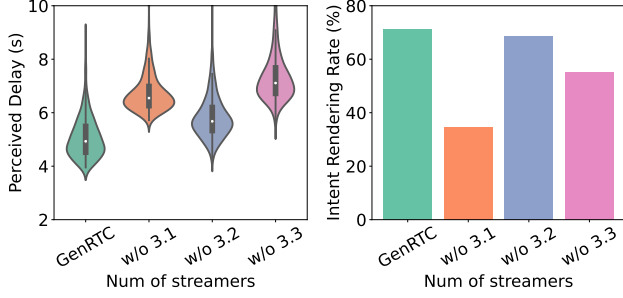


Figure 14: The perceived delay and GPU utility rate of GenRTC when each design module is removed.

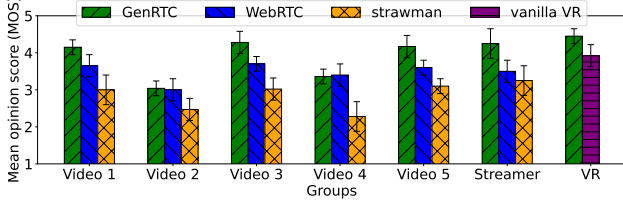


Figure 15: The user preference in MOS for live streaming audiences, streamers making free speech, and VR game players.

module in GenRTC architecture.

5.5 Case Study: Live Video Streaming

To gain qualitative insights into user experiences in live video streaming, we conducted a user study involving 20 volunteers from diverse background. Participants interacted with GenRTC through standard streaming scenarios, with 5 of them also participating as streamers, giving free talks and watching their own echo video in real time, which is common practice for streamers. Volunteers used three types of devices including laptops, desktops, and smartphones, to access GenRTC via the Web. For the audience, we played 5 randomly selected videos from the dataset, covering three categories: sports events, talk shows, and variety shows. Volunteers experienced all three schemes in random order and rated each on a 5-point scale (1=very bad, 2=bad, 3=fair, 4=good, 5=very good).

Figure 15 presents the results. Their feedback highlights the practical impact of our technological enhancements and helps gauge user satisfaction in real-world conditions, making it a crucial part of our system evaluation. An additional discovery is that, from a linguistic perspective, visual captions influence the speaking patterns of streamers. For example, streamers may reference the visual captions to continue their speech. This represents a key distinction between trace-driven simulations and real user studies.

5.6 Case Study: Collaborative VR Scenario

Collaborative VR offers unique opportunities for GenRTC, especially due to its immersive nature and the crucial need for

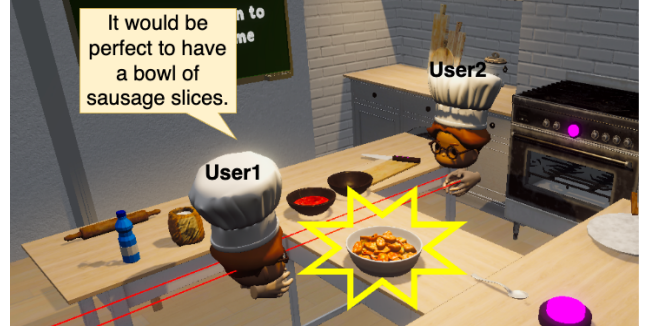
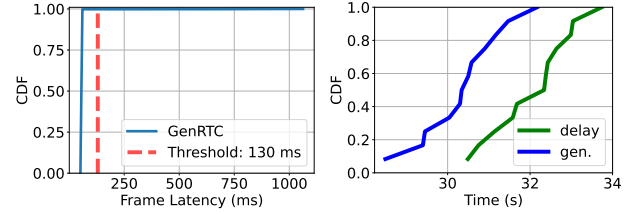


Figure 16: The 3D generation effect in the VR collaborative game [47]. The generated object is highlighted.



(a) Frame latency: 99.18% of (b) Perceived delay and generation measurements are <65ms.

Figure 17: The frame latency and perceived latency of 3D generation with Rodin in VR collaboration game.

3D innovations and creations [9, 41]. However, the generated objects need to be 3D to be compatible with the environments, which poses a more harsh challenge compared to images. This case study explores the application of GenRTC in VR scenarios, evaluating its performance in terms of frame latency and perceived delay.

Setup. The evaluation takes place in a controlled VR environment where two participants engage in an interactive session. Users operate Meta Oculus 2 [32] headsets, connected to the server via USB, with Netcode [54] applied to simulate a one-way latency of 50 ms. Since there are no specific frame definitions in VR volumetric streaming, we measure perceived delay and frame latency using the Unity API [54]. The two users form a 1-on-1 conferencing structure, where both act as streamers and audience members. However, the upstream consists of only sparse command flow within the game. Additionally, the object is separated from the VR stream, so the renderer component is disabled.

Model Selection. Text-to-3D models are computationally intensive for both training and inference. We test the pre-trained models Prolificdreamer [57] and DreamGaussian [68] on our server, as well as the closed-source model Rodin-Gen-1 [56] via API calls. The first two models take over 2 minutes to run, while Rodin completes the generation in approximately 30 seconds. Due to its superior performance, we selected Rodin for the following experiments. The rendered mesh and texture are stored in the Unity project and automatically

activated for display. As shown in Figure 16, user 1 expresses the need for a bowl of sausage slices, triggering the GenRTC workflow with the Rodin-Gen-1 model.

Through the user study, 4 users form 2 pairs, with 24 objects generated. Figure 17 shows that 99.18% of frame delay is less than 65ms, because only the packets for sparse human movements and audio are transmitted. However, when the generated object first occurs, there is a traffic burst due to bulk transfer, in which the average frame latency is 271.4ms. The perceived delay for this case is 32.1 seconds, with 30.4 seconds spent on generation inference. This result demonstrates that, in 3D object generation, the majority of the delay is due to the generation process rather than to system flaws.

This case study underscores the potential of GenRTC to enhance interactive real-time applications in VR, suggesting avenues for further research and development in this rapidly evolving field. This structured assessment allows us to validate the effectiveness of GenRTC under various quality metrics, ensuring that the system’s performance is consistent and reliable regardless of the input quality.

6 Related Work

Latency Optimization for Live Video Streaming. Predictive content can boost timeliness. ZGaming [62] reduces the latency of cloud gaming by predicting the foreground of the game scenes. Farfetchfusion [19] uses a disentangled fusion approach and topology anchoring to overcome the challenges of 3D reconstruction latency and computational complexity on mobile devices. Super-resolution is another popular machine learning pipeline for multimedia transport systems [13]. However, most of them are performed on the edges and therefore do not raise latency issues [49]. The CV domain also highlights generation speed and efficiency to approach the real-world live streaming needs [15], but the generation time is far from acceptable. Nevertheless, GenRTC will benefit from any future optimizations on generative models.

Generative Models for Visual Object. The generation can be categorized into three major kinds: object, scene, and human [22]. Rodin [56] and HeadNeRF [12] generate human head avatars. DreamFusion [42] utilizes a pre-trained 2D text-to-image diffusion model to perform general text-to-3D synthesis. Prolificdreamer [57] refines score-distillation sampling to generate high-resolution 3D objects. Other works explore the systematical challenges of adopting 3D generation, especially in edges and mobile devices. Zhang et al. [72] explores how to enable text-3D generation on resource-constrained mobile devices. CoDi [26] and GaussianDreamer [68] improve generation speed, but it still takes more than 2 minutes. DiffAgent [75] selects the best text-to-image API in seconds to better match needs, but this selection latency is unacceptable. Our work pushes a steady step towards its application. Future works can explore the follow-up effects [63] for generated objects to maintain positional consistency during interactions.

GPU Resource Scheduling. The efficient scheduling of limited GPU resources for multiple sessions is critical [35, 50, 58, 67]. For example, Gravel [35] assigns jobs to heterogeneous GPUs. Usher [50] enhances GPU utilization for ML inference by optimizing resource allocation and reducing inter-model interference. CacheSack [67] improves flash cache efficiency by optimizing admission policies, reducing disk reads, flash writes, and overall costs. Zhao et al. [74] train job placement by optimizing resource allocation with statistical INA, improving efficiency and job completion times. Following the optimization pattern of these works, GenRTC formulates GPU scheduling as an online knapsack problem and applies the heuristics from LLMs to form an MPC-based solution to maximize the expectation of intent rendering rate.

Compatibility with Recent Live Streaming Advances. GenRTC is compatible with recent advances in neural compression and loss recovery frameworks, as mentioned below: Gemino [52] applies neural compression for video conferencing, reducing bandwidth usage while maintaining high-quality video at low bitrates. Tambur [45] improves videoconferencing by using streaming codes for efficient loss recovery, reducing freezes and bandwidth overhead. Grace [5] optimizes video transmission by optimizing neural codecs for loss recovery, reducing undecodable frames and bandwidth usage. For these works, GenRTC may transmit the visual captions in a side datachannel and combine with the frames after decoding. Hairpin [28] enhances interactive video streaming by optimizing retransmissions and redundancy, reducing bandwidth costs and deadline misses. The frame rate adaptation is proposed by AFR [29] to optimize tail latency by introducing the frame rate adaptation mechanism. Yuzu [70] improves volumetric video streaming with optimized 3D super-resolution. MuV2 [24] enhances multi-user volumetric video streaming by optimizing resource use through content hybridization, view sharing, and encoder multiplexing. GenRTC can compatibly benefit from these innovations to further reduce the latency. Overall, there is a huge systematical gap between these novel generating functionalities and product-level adoption, which this work fills.

7 Conclusion

This paper proposes GenRTC, a system that advances real-time video streaming experience by seamlessly integrating visual captions into live video streams. By continuously analyzing the speech intents, GenRTC translates speech into visual representations in real-time, enhancing both user engagement and comprehension. The system’s robust handling of latency, scalability, and resource management ensures it meets the stringent demands of live streaming environments. Extensive evaluation on dataset-driven experiments, user studies and VR case studies shows that GenRTC can achieve a high level of efficiency and responsiveness, providing the captions without compromising latency and video quality.

References

- [1] Devar uses generative AI to create 3D AR images from text prompts. <https://venturebeat.com/games/devar-uses-generative-ai-to-create-3d-ar-images-from-text-prompts>.
- [2] aiortc. aiortc. <https://github.com/aiortc/aiortc>.
- [3] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. Analysis and design of the google congestion control for web real-time communication (webrtc). In *Proceedings of the 7th International Conference on Multimedia Systems*, pages 1–12, 2016.
- [4] Jiayi Chen, Nihal Sharma, Tarannum Khan, Shu Liu, Brian Chang, Aditya Akella, Sanjay Shakkottai, and Ramesh K Sitaraman. Darwin: Flexible learning-based cdn caching. In *ACM SIGCOMM*, 2023.
- [5] Yihua Cheng, Ziyi Zhang, Hanchen Li, Anton Arapin, Yue Zhang, Qizheng Zhang, Yuhao Liu, Kuntai Du, Xu Zhang, Francis Y Yan, et al. GRACE: Loss-Resilient Real-Time Video through Neural Codecs. In *USENIX NSDI*, 2024.
- [6] Sheng-Yang Chiu, Yu-Ting Huang, Chieh-Ting Lin, Yu-Chee Tseng, Jen-Jee Chen, Meng-Hsuan Tu, Bo-Chen Tung, and Yu-Jou Nieh. Privacy-preserving video conferencing via thermal-generative images. In *IEEE ICRA*. IEEE, 2023.
- [7] Federal Communication Commission. Measuring Broadband America - July 2012. <https://www.fcc.gov/reports-research/reports/measuring-broadband-america/measuring-broadband-america-july-2012>.
- [8] Eduardo Cuervo, Alec Wolman, Landon P Cox, Kiron Lebeck, Ali Razeen, Stefan Saroiu, and Madanlal Musuvathi. Kahawai: High-quality mobile gaming using gpu offload. In *Proceedings of the 13th annual international conference on mobile systems, applications, and services*, pages 121–135, 2015.
- [9] Mallesh Dasari, Edward Lu, Michael W Farb, Nuno Pereira, Ivan Liang, and Anthony Rowe. Scaling VR Video Conferencing. In *IEEE VR*. IEEE, 2023.
- [10] Sandra Götz. Fluency in native and nonnative English speech. 2013.
- [11] GpuMart. Dedicated A100 GPU Hosting, NVIDIA A100 Rental. <https://www.gpu-mart.com/nvidia-a100-rental#plans>.
- [12] Yang Hong, Bo Peng, Haiyao Xiao, Ligang Liu, and Juyong Zhang. Headerf: A real-time nerf-based parametric head model. In *IEEE/CVF CVPR*, 2022.
- [13] Shuowei Jin, Ruiyang Zhu, Ahmad Hassan, Xiao Zhu, Xumiao Zhang, Z Morley Mao, Feng Qian, and Zhi-Li Zhang. OASIS: Collaborative Neural-Enhanced Mobile Video Streaming. In *ACM MMSYS*, 2024.
- [14] Alan Jones, Peter Sevcik, and Rebecca Wetzel. Internet connection requirements for effective video conferencing to support work from home and eLearning. *NetForecast Report*, 2021.
- [15] Ozgur Kara, Bariscan Kurtkaya, Hidir Yesiltepe, James M. Rehg, and Pinar Yanardag. RAVE: Randomized Noise Shuffling for Fast and Consistent Video Editing with Diffusion Models. In *IEEE/CVF CVPR*, 2024.
- [16] kubernetes. Production-Grade Container Orchestration. <https://kubernetes.io/>.
- [17] Lambda. Introducing NVIDIA RTX A6000 GPU Instances on Lambda Cloud. <https://lambdalabs.com>.
- [18] Chan C Lee and Der-Tsai Lee. A simple on-line bin-packing algorithm. *Journal of the ACM (JACM)*, 32(3):562–572, 1985.
- [19] Kyungjin Lee, Juheon Yi, and Youngki Lee. Farfetch-Fusion: Towards Fully Mobile Live 3D Telepresence Platform. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, 2023.
- [20] Xinyu Li, Chuang Zhao, Hongke Zhao, Likang Wu, and Ming HE. GANPrompt: Enhancing Robustness in LLM-Based Recommendations with GAN-Enhanced Diversity Prompts. *arXiv preprint arXiv:2408.09671*, 2024.
- [21] Feifan Liu, Deana Pennell, Fei Liu, and Yang Liu. Unsupervised approaches for automatic keyword extraction using meeting transcripts. In *ACM NAACL*, 2009.
- [22] Jian Liu, Xiaoshui Huang, Tianyu Huang, Lu Chen, Yue-nan Hou, Shixiang Tang, Ziwei Liu, Wanli Ouyang, Wangmeng Zuo, Junjun Jiang, et al. A Comprehensive Survey on 3D Content Generation. *arXiv preprint arXiv:2402.01166*, 2024.
- [23] Xingyu Liu, Vladimir Kirilyuk, Xiuxiu Yuan, Peggy Chi, Xiang Chen, Alex Olwal, and Ruofei Du. Visual Captions: Augmenting Verbal Communication with On-the-Fly Visuals. In *ACM CHI*, 2023.
- [24] Yu Liu, Puqi Zhou, Zejun Zhang, Anlan Zhang, Bo Han, Zhenhua Li, and Feng Qian. MuV2: Scaling up Multi-user Mobile Volumetric Video Streaming via Content Hybridization and Sharing. In *ACM MOBICOM*, 2024.
- [25] Douglas W Maynard. Placement of topic changes in conversation. 1980.

- [26] Kangfu Mei, Mauricio Delbracio, Hossein Talebi, Zhengzhong Tu, Vishal M Patel, and Peyman Milanfar. CoDi: Conditional Diffusion Distillation for Higher-Fidelity and Faster Image Generation. In *IEEE/CVF CVPR*, 2024.
- [27] Zili Meng, Yaning Guo, Chen Sun, Bo Wang, Justine Sherry, Hongqiang Harry Liu, and Mingwei Xu. Achieving consistent low latency for wireless real-time communications with the shortest control loop. In *ACM SIGCOMM*, 2022.
- [28] Zili Meng, Xiao Kong, Jing Chen, Bo Wang, Mingwei Xu, Rui Han, Honghao Liu, Venkat Arun, Hongxin Hu, and Xue Wei. Hairpin: Rethinking packet loss recovery in edge-based interactive video streaming. In *USENIX NSDI*, 2024.
- [29] Zili Meng, Tingfeng Wang, Yixin Shen, Bo Wang, Mingwei Xu, Rui Han, Honghao Liu, Venkat Arun, Hongxin Hu, and Xue Wei. Enabling high quality Real-Time communications with adaptive Frame-Rate. In *USENIX NSDI*, 2023.
- [30] Meta. Livestream from Meta Quest to YouTube using OBS. <https://www.meta.com/help/quest/articles/in-vr-experiences/social-features-and-sharing/livestream-to-youtube-obs/>.
- [31] Meta. Meet Llama 3.1. <https://www.llama.com/>.
- [32] Meta. Meta Quest 2. <https://www.meta.com/quest/products/quest-2/>.
- [33] Arsha Nagrani, Joon Son Chung, and Andrew Zisserman. Voxceleb: a large-scale speaker identification dataset. *arXiv preprint arXiv:1706.08612*, 2017.
- [34] Jisu Nam, Heesu Kim, DongJae Lee, Siyoon Jin, Seungryong Kim, and Seunggyu Chang. Dreammatcher: Appearance matching self-attention for semantically-consistent text-to-image personalization. In *IEEE/CVF CVPR*, 2024.
- [35] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *USENIX OSDI*, 2020.
- [36] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. Mahimahi: accurate {Record-and-Replay} for {HTTP}. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 417–429, 2015.
- [37] Ilya Nikolaevskiy. Refactor FrameBuffer to allow for new decode scheduling strategy. <https://issues.webrtc.org/issues/42223541>.
- [38] NVIDIA. Inference Performance of NVIDIA Data Center Products. <https://developer.nvidia.com/deep-learning-performance-training-inference/ai-inference>.
- [39] OpenAI. DALL-E 2. <https://openai.com/index/dall-e-2/>.
- [40] OpenAI. DALL-E 3. <https://openai.com/index/dall-e-3/>.
- [41] Gustav Bøg Petersen, Valdemar Stenberdt, Richard E Mayer, and Guido Makransky. Collaborative generative learning activities in immersive virtual reality increase learning. *Computers & Education*, 207:104931, 2023.
- [42] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. DreamFusion: Text-to-3D using 2D Diffusion. In *ICLR*.
- [43] Tingting Qiao, Jing Zhang, Duanqing Xu, and Dacheng Tao. MirrorGAN: Learning Text-To-Image Generation by Redescription. In *IEEE/CVF CVPR*, June 2019.
- [44] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF CVPR*, 2022.
- [45] Michael Rudow, Francis Y Yan, Abhishek Kumar, Ganesh Ananthanarayanan, Martin Ellis, and KV Rashmi. Tambur: Efficient loss recovery for videoconferencing via streaming codes. In *USENIX NSDI*, 2023.
- [46] Stability AI Runway, CompVis. Stability AI. <https://stability.ai/stable-image>.
- [47] Javad Sameri, Sam Van Damme, Susanna Schwarzmann, Qing Wei, Riccardo Trivisonno, Filip De Turck, and Maria Torres Vega. Collaborative Cooking in VR: Effects of Network Distortion in Multi-User Virtual Environments. In *ACM MMSYS*, 2024.
- [48] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *ECCV*, 2025.
- [49] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE internet of things journal*, 3(5):637–646, 2016.
- [50] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. USHER: Holistic Interference Avoidance for Resource Optimized ML Inference. In *USENIX OSDI*, 2024.
- [51] Anirudh Sivaraman, Suvinay Subramanian, Mohammad Alizadeh, Sharad Chole, Shang-Tse Chuang, Anurag Agrawal, Hari Balakrishnan, Tom Edsall, Sachin Katti,

- and Nick McKeown. Programmable packet scheduling at line rate. In *SIGCOMM*, 2016.
- [52] Vibhaalakshmi Sivaraman, Pantea Karimi, Vedantha Venkatapathy, Mehrdad Khani, Sadjad Fouladi, Mohammad Alizadeh, Frédo Durand, and Vivienne Sze. Gemino: Practical and Robust Neural Compression for Video Conferencing. *arXiv preprint arXiv:2209.10507*, 2022.
- [53] Aaron Ullal. What powers Google Meet and Microsoft Teams? WebRTC Demystified. <https://levelup.gitconnected.com/what-powers-google-meet-and-microsoft-teams-webrtc-demystified-step-by-step-tutorial-e0cb422010f7>.
- [54] Unity. Managing latency with prediction. <https://docs.unity3d.com/Packages/com.unity.netcode@1.3/manual/prediction-n4e.html>.
- [55] Talha Waheed, Ihsan Ayyub Qazi, Zahaib Akhtar, and Zafar Ayyub Qazi. Coal not diamonds: how memory pressure falters mobile video QOE. In *ACM CONEXT*, 2022.
- [56] Tengfei Wang, Bo Zhang, Ting Zhang, Shuyang Gu, Jianmin Bao, Tadas Baltrusaitis, Jingjing Shen, Dong Chen, Fang Wen, Qifeng Chen, et al. Rodin: A generative model for sculpting 3d digital avatars using diffusion. In *IEEE/CVF CVPR*, 2023.
- [57] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *Advances in Neural Information Processing Systems*, 36, 2024.
- [58] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y Yan. Autothrottle: A Practical {Bi-Level} Approach to Resource Management for {SLO-Targeted} Microservices. In *USENIX NSDI 24*, 2024.
- [59] WebRTC. WebRTC Cloud Gaming: Unboxing Stadia. <https://webrtc.ventures/2021/02/webrtc-cloud-gaming-unboxing-stadia/>.
- [60] Mike Wong, Murali Ramanujam, Guha Balakrishnan, and Ravi Netravali. MadEye: Boosting Live Video Analytics Accuracy with Adaptive Camera Configurations. In *USENIX NSDI*, 2024.
- [61] Bingyang Wu, Kun Qian, Bo Li, Yunfei Ma, Qi Zhang, Zhigang Jiang, Jiayu Zhao, Dennis Cai, Ennan Zhai, Xuanzhe Liu, et al. Xron: A hybrid elastic cloud overlay network for video conferencing at planetary scale. In *ACM SIGCOMM*, 2023.
- [62] Jiangkai Wu, Yu Guan, Qi Mao, Yong Cui, Zongming Guo, and Xinggong Zhang. ZGaming: Zero-latency 3D cloud gaming by image prediction. In *ACM SIGCOMM 2023*, 2023.
- [63] Jingao Xu, Guoxuan Chi, Zheng Yang, Danyang Li, Qian Zhang, Qiang Ma, and Xin Miao. Followupar: Enabling follow-up effects in mobile ar applications. In *ACM MOBISYS*, 2021.
- [64] Shuyuan Xu, Jun Wang, Wenchi Shou, Tuan Ngo, Abdul-Manan Sadick, and Xiangyu Wang. Computer vision techniques in construction: a critical review. *Archives of Computational Methods in Engineering*, 28:3383–3397, 2021.
- [65] Francis Y Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *USENIX NSDI 20*, 2020.
- [66] Juncheng Yang, Anirudh Sabnis, Daniel S Berger, KV Rashmi, and Ramesh K Sitaraman. C2DN: How to Harness Erasure Codes at the Edge for Efficient Content Delivery. In *USENIX NSDI*, 2022.
- [67] Tzu-Wei Yang, Seth Pollen, Mustafa Uysal, Arif Merchant, and Homer Wolfmeister. CacheSack: Admission Optimization for Google Datacenter Flash Caches. In *USENIX ATC*, 2022.
- [68] Taoran Yi, Jiemin Fang, Junjie Wang, Guanjun Wu, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Qi Tian, and Xinggong Wang. GaussianDreamer: Fast Generation from Text to 3D Gaussians by Bridging 2D and 3D Diffusion Models. In *IEEE/CVF CVPR*, 2024.
- [69] YouTube. Use automatic captioning. <https://support.google.com/youtube/answer/6373554>.
- [70] Anlan Zhang, Chendong Wang, Bo Han, and Feng Qian. YuZu: Neural-enhanced Volumetric Video Streaming. In *USENIX NSDI*, 2022.
- [71] Xu Zhang, Yiyang Ou, Siddhartha Sen, and Junchen Jiang. {SENSEI}: Aligning video streaming quality with dynamic user sensitivity. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 303–320, 2021.
- [72] Xuechen Zhang, Zheng Li, Samet Oymak, and Jiasi Chen. Text-to-3D Generative AI on Mobile Devices: Measurements and Optimizations. In *ACM SIGCOMM*, 2023.
- [73] Zhixing Zhang, Ligong Han, Arnab Ghosh, Dimitris N Metaxas, and Jian Ren. Sine: Single image editing with text-to-image diffusion models. In *Proceedings of the*

IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023.

- [74] Bohan Zhao, Wei Xu, Shuo Liu, Yang Tian, Qiaoling Wang, and Wenfei Wu. Training Job Placement in Clusters with Statistical In-Network Aggregation. In *ACM ASPLOS*, 2024.
- [75] Lirui Zhao, Yue Yang, Kaipeng Zhang, Wenqi Shao,

Yuxin Zhang, Yu Qiao, Ping Luo, and Rongrong Ji. DiffAgent: Fast and Accurate Text-to-Image API Selection with Large Language Model. In *IEEE/CVF CVPR*, 2024.

- [76] Zoom. Bringing Zoom’s end-to-end optimizations to WebRTC. <https://blog.livekit.io/livekit-one-dot-zero/>.